1 Multiplicative contraction

Source: `PrimeTensor/Tensor/Contract.lean`

What this file establishes

The multiplicative trace of a square matrix: the product of its diagonal entries. Where the classical trace sums the diagonal, this multiplies it, which is the right move for a carrier whose primitive operation is multiplication and which offers no addition.

The file is three declarations, and its interest is that it needs no empty case. A product over an index set must return something when that set is empty, and the only uniform answer is the unit of the operation — so a contraction defined over Fin n would have to demand a unit from the coefficient type and single out $n = 0$. Here the dimension is a depth and therefore at least one, the fold that computes the product needs no identity element, and the coefficient type is asked only for a multiplication.

1.1 The definition

Definition 1.1 (Contraction, `Tensor.contract₂`)**.** For a type A with a multiplication and m : Matrix A *dim*,

$$\mathrm{contract}_2(m) \equiv \mathrm{fold}_{\cdot}\big(\mathit{dim},\ \lambda i.\ m.\mathrm{component}\ (i,i)\big)\ :\ A.$$

```
def contract₂ {A : Type} [Mul A] {dim : Depth} (m : Matrix A dim) : A :=
  Axis.fold (· * ·) dim (fun i => m.component (i, i))
```

Two things from earlier modules are being used without ceremony. The diagonal selector $\lambda i.\ m.\mathrm{component}\ (i,i)$ is well typed because at rank two the index type *is* a product of two axis types, definitionally, as established in `PrimeTensor/Tensor/Basic.lean`. And Axis.fold, from `PrimeTensor/Depth.lean`, folds a function on Axis *dim* with a binary operation and no seed value.

Proposition 1.2 (What the contraction is)**.** *Let $n = |\mathit{dim}|$ and let $i_1, \ldots, i_n$ enumerate* Axis *dim in order. Writing $m_{jk} = m.\mathrm{component}\ (j,k)$,*

$$\mathrm{contract}_2(m) = m_{i_1 i_1} \cdot (m_{i_2 i_2} \cdot (\cdots \cdot m_{i_n i_n})),$$

the right-nested product of the diagonal entries in axis order, with no unit at the innermost position.

Proof. Immediate from Definition 1.1 and the expansion of Axis.fold proved in `PrimeTensor/Depth.lean`, applied to the diagonal selector. □

Design note 1.3 (Only a magma, and the bracketing is real). The instance argument is `[Mul A]` alone: no associativity, no commutativity, no unit. Two consequences deserve stating, because for the classical trace neither arises.

First, the bracketing in Proposition 1.2 is part of the value, not an artefact of how it is written. Without associativity, a differently bracketed product of the same diagonal entries need not be the same element of A.

Second, the *order* of the diagonal entries is likewise part of the value, fixed by the enumeration of Axis *dim*. Without commutativity there is no permutation invariance to appeal to. For the

intended carriers — strictly positive reals under multiplication — both laws do hold, so nothing is lost there; the definition simply does not assume them, and so applies to carriers where they fail.

1.2 The two computation rules

Lemma 1.4 (Base case, `contract₂_dim_one`)**.** *For* m : Matrix A one*,*

$$\mathrm{contract}_2(m) = m.\mathrm{component}\ (\mathsf{first}, \mathsf{first}).$$

Lemma 1.5 (Successor case, `contract₂_dim_succ`)**.** *For* m : Matrix A ($\mathsf{succ}\, d$)*,*

$$\mathrm{contract}_2(m) = m.\mathrm{component}\ (\mathsf{first}, \mathsf{first}) \cdot \mathrm{fold}_{\cdot}(d,\ \lambda i.\ m.\mathrm{component}\ (\mathsf{next}\, i, \mathsf{next}\, i)).$$

```
@[simp] theorem contract₂_dim_one {A : Type} [Mul A]
    (m : Matrix A .one) :
    contract₂ m = m.component (.first, .first) := rfl

@[simp] theorem contract₂_dim_succ {A : Type} [Mul A]
    {d : Depth} (m : Matrix A (.succ d)) :
    contract₂ m =
      m.component (.first, .first) *
        Axis.fold (· * ·) d (fun i => m.component (.next i, .next i)) := rfl
```

Proof. Both are rfl. Unfolding Definition 1.1 turns each side into an application of Axis.fold at a dimension in constructor form, where the corresponding defining equation of fold — fold_one and fold_succ in `PrimeTensor/Depth.lean` — applies definitionally. As there, the statements are content-free as propositions and the `@[simp]` tags are the point: they let the simplifier evaluate a contraction whose dimension is a concrete numeral, which unfolding $\mathrm{contract}_2$ alone would not do. Together they are the complete case analysis on the dimension, so no third rule is possible. □

Remark 1.6 (The successor rule is not recursive). Lemma 1.5 exposes a raw Axis.fold on its right-hand side rather than a smaller contraction, and it has to: m has type Matrix A ($\mathsf{succ}\, d$), whose component function is defined on Axis ($\mathsf{succ}\, d$), and there is no matrix of dimension d sitting inside it to contract.

One can of course build the restriction along next,

$$m' \equiv \langle\, \lambda ij.\ m.\mathrm{component}\ (\mathsf{next}\, ij_1, \mathsf{next}\, ij_2)\, \rangle\ :\ \mathrm{Matrix}\ A\ d,$$

and then $\mathrm{contract}_2(m')$ is definitionally the fold appearing in the lemma, so the rule could equally have been phrased recursively. The file does not introduce m'. The practical consequence is that an induction on the dimension about $\mathrm{contract}_2$ is carried out at the level of Axis.fold, generalising over the selector function, rather than by recursion on contractions. (m' is not a declaration of the file; it is written here only to say what the successor rule amounts to.)

Remark 1.7 (Scope). Only rank two is treated, as the subscript in the name records: there is no contraction of a general tensor, no contraction of a chosen pair of indices, and no interaction with the outer product of `PrimeTensor/Tensor/Basic.lean` — in particular nothing here relates $\mathrm{contract}_2(\mathrm{outer}(u, v))$ to the coefficients of u and v. Nor is anything proved about $\mathrm{contract}_2$ beyond its two computation rules.

Summary of the correspondence

Lean declaration	Kind	Here
`Tensor.contract₂`	definition	Def. 1.1
`Tensor.contract₂_dim_one`	simp thm	Lem. 1.4
`Tensor.contract₂_dim_succ`	simp thm	Lem. 1.5

Every declaration of `PrimeTensor/Tensor/Contract.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition; its two theorems are proved by `rfl`. Proposition 1.2 and Remark 1.6 are stated for the reader and are not declarations of the file.